

SET Important Questions for MSE 2(U3 & U5)

1. Explain Different types of performance testing

1. Load Testing

System is tested with **expected number of users**. It checks if it works smoothly under normal conditions.



Example:

An online shopping site is tested with **1,000 users** browsing and purchasing at the same time. If response time stays within limit (e.g., 3 seconds), it passes.

2. Stress Testing

System is tested **beyond its capacity** to find the breaking point.

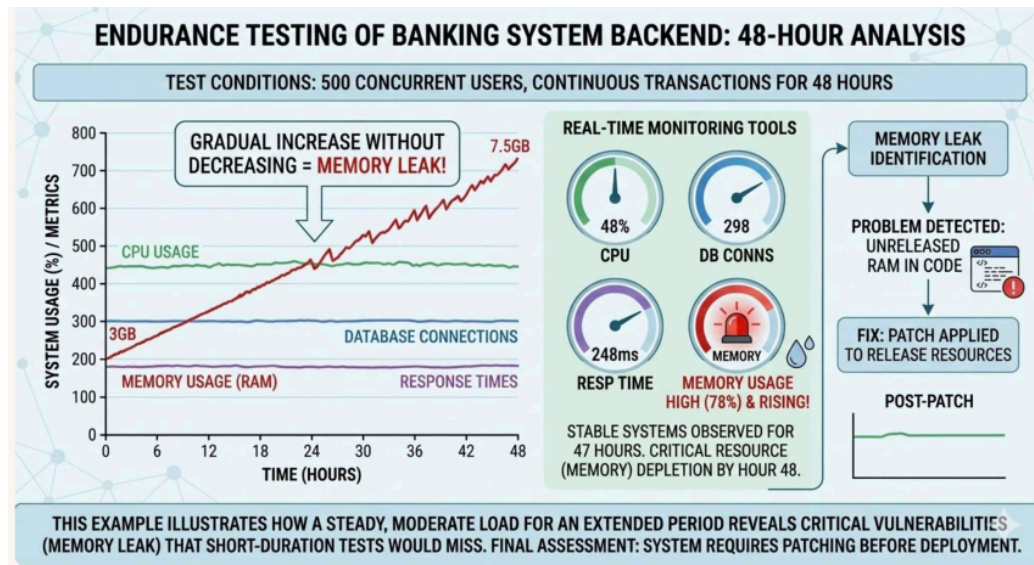


Example:

Same website is tested with **5,000 users** when it is designed for 1,000. System may slow down, show errors, or crash—this shows its limit.

3. Endurance (Soak) Testing

System is tested for a **long time with steady load** to detect issues like memory leaks.

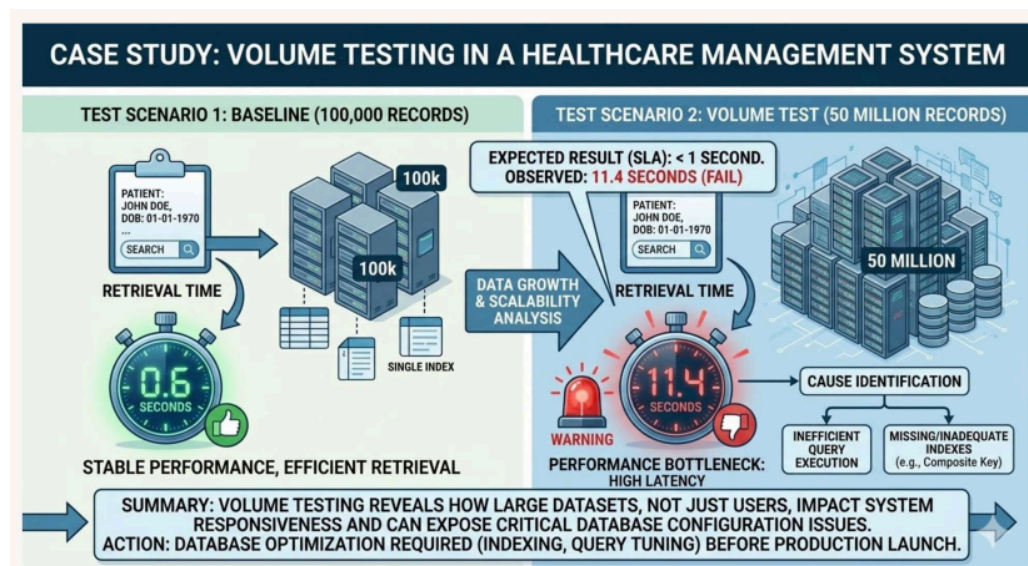


Example:

A banking system runs continuously with **500 users for 48 hours**. If memory keeps increasing without releasing, it indicates a problem.

4. Volume (Flood) Testing

System is tested with **huge amount of data** instead of users.



Example:

Database grows from **1 lakh records to 5 crore records**, and queries are tested. If data retrieval becomes slow, performance issue is identified.

2. CID Triad

CIA Triad (Confidentiality, Integrity, Availability)

It is the **core foundation of security** used to protect systems and data. It defines three essential goals that every secure system must satisfy.

1. Confidentiality

Ensures that **data is not exposed to unauthorized users**.

It focuses on privacy and secrecy of information.

Example:

In a hospital system, patient medical records are accessible only to doctors and authorized staff, not to everyone.

2. Integrity

Ensures that **data remains correct, consistent, and unaltered**.

It prevents unauthorized modification of data.

Example:

If a student's marks are stored in a database, they should not be changed by anyone except authorized faculty.

3. Availability

Ensures that **systems and data are available whenever required**.

It focuses on uninterrupted access to services.

Example:

An ATM system should be available 24/7 so users can withdraw money anytime.

Together, these three ensure that a system is **secure, reliable, and trustworthy**.

3. STRIDE

STRIDE (Security Threat Model)

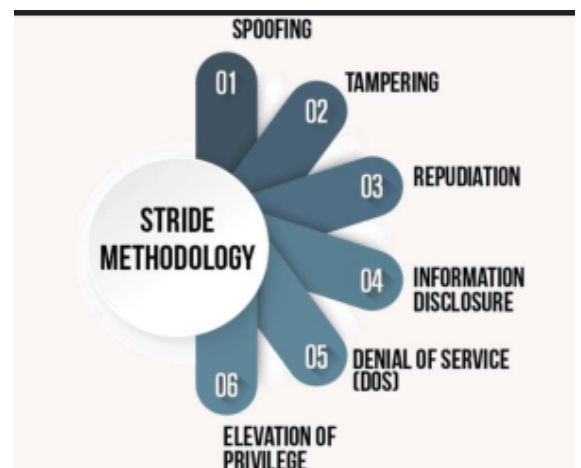
STRIDE is a model used to identify different types of **security threats in a system**. Each letter represents one category of threat.

S – Spoofing (Identity Threat)

Pretending to be someone else to gain access.

Example:

An attacker logs in using stolen username and password of another user.



T – Tampering (Data Threat)

Unauthorized modification of data.

Example:

Changing transaction details like amount during money transfer.

R – Repudiation (Denial of Action)

User denies performing an action without proof.

Example:

A user makes an online payment and later denies it.

I – Information Disclosure (Privacy Threat)

Exposure of sensitive information to unauthorized users.

Example:

Leaking user passwords or personal data from a database.

D – Denial of Service (DoS)

Making the system unavailable to users.

Example:

Flooding a website with too many requests so it crashes.

E – Elevation of Privilege

Gaining higher access rights than permitted.

Example:

A normal user exploits a bug and gets admin privileges.

4.OWASP

OWASP (Open Web Application Security Project)

OWASP is an **organization focused on improving the security of web applications** by identifying common vulnerabilities and providing guidelines to fix them.

OWASP Top Vulnerabilities (with examples):

1. Injection

A login query:

```
SELECT * FROM users
WHERE username='admin'
AND password='password'
```

If an attacker enters:

```
' OR 1=1 --
```

The condition becomes true and login may succeed.

Attacker injects malicious code into queries.

Example: SQL Injection – entering `' OR 1=1 --` in login to bypass authentication.

2. Broken Authentication

Example passwords allowed:

```
123456
password
admin
```

Weak login or session management.

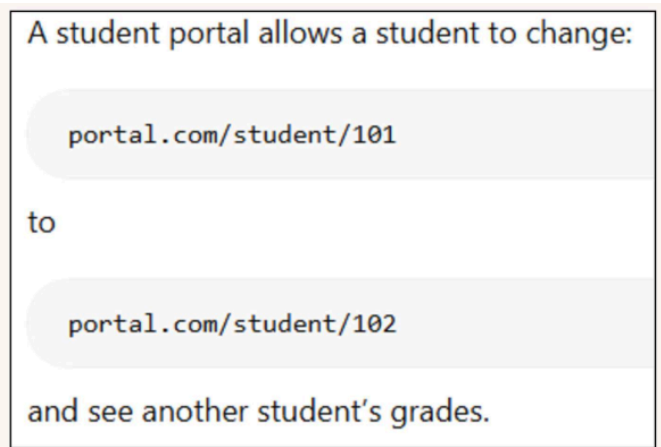
Example: Using simple passwords like “1234” allows attackers to access accounts.

3. Sensitive Data Exposure

Failure to protect confidential data.

Example: Storing passwords in plain text instead of encryption.

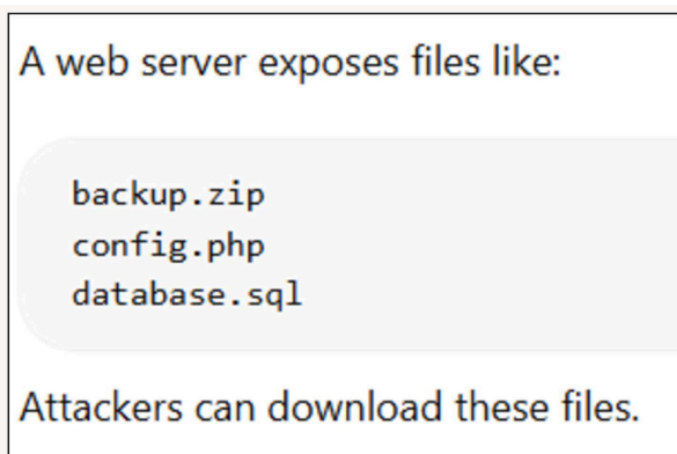
4. Broken Access Control



Users can access unauthorized resources.

Example: Normal user accessing admin page by changing URL.

5. Security Misconfiguration



Improper system settings.

Example: Leaving default passwords or open ports.

6. Cross-Site Scripting (XSS)

Injecting malicious scripts into web pages.

Example: Attacker inserts JavaScript to steal user cookies.

7. Insecure Deserialization

Unsafe data handling leading to attacks.

Example: Manipulating serialized data to execute harmful code.

8. Using Components with Known Vulnerabilities

Using outdated libraries.

Example: Old framework with known security bugs.

9. Insufficient Logging & Monitoring

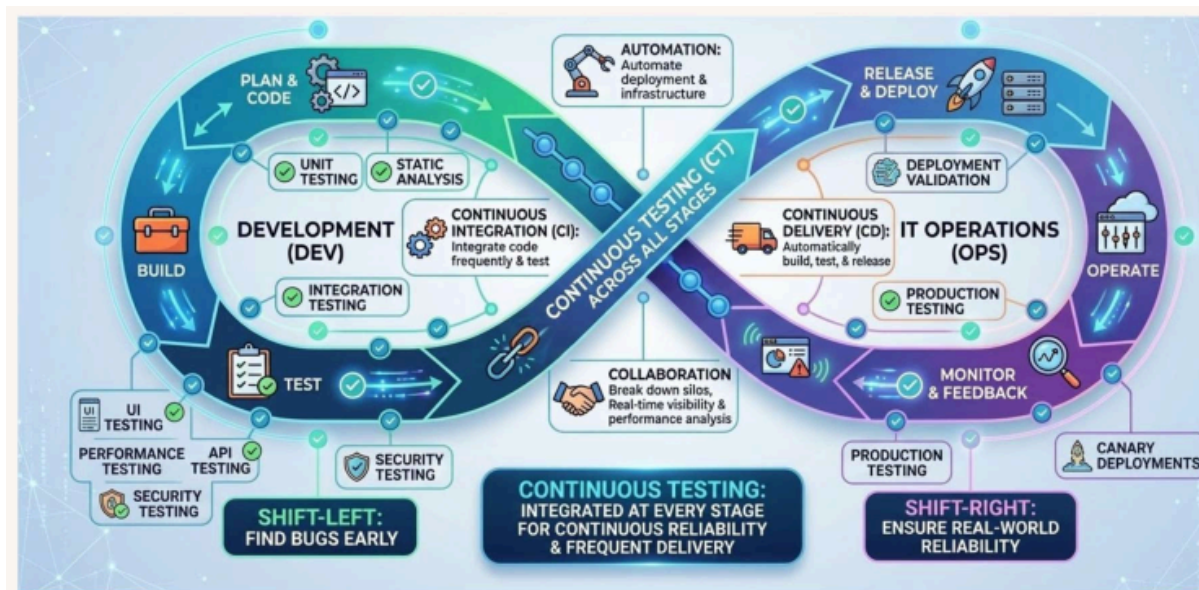
Lack of tracking security events.

Example: Unauthorized login attempts not recorded.

OWASP helps developers **identify, prevent, and fix** these vulnerabilities to build secure applications.

5. Shift left, Shift Right

Shift Left & Shift Right (in Security/Testing)



1. Shift Left

Testing and security activities are moved **early in the development process** (before coding or during coding).

Explanation:

Instead of finding bugs after development, issues are identified at requirements or design stage itself. This reduces cost and improves quality.

Example:

While developing a login feature, developers check for security issues (like input validation, password rules) during coding itself instead of waiting for final testing.

2. Shift Right

Testing and monitoring are done **after deployment in production**.

Explanation:

Focuses on real-world usage, monitoring performance, user behavior, and catching issues that appear only in live environments.

Example:

After deploying an app, monitoring tools track crashes, response time, and user errors. If users face login failures, it is detected and fixed quickly.

6. Software Development life cycle (Any 1)

Software Development Life Cycle (SDLC)

SDLC is a **systematic process used to develop software step by step**, from initial idea to maintenance. It helps in planning, building, testing, and delivering high-quality software.

Phases of SDLC (5-mark explanation):

1. Feasibility Study

This phase analyzes whether the project is **practical and achievable**. It includes cost, time, and technical feasibility.

Example:

Checking if building a food delivery app is possible within budget and time.

2. Requirements Analysis & Specification

All user requirements are **collected, analyzed, and documented clearly**.

Example:

User should be able to register, order food, and track delivery.

3. Design

System architecture and structure are **designed based on requirements**.

Example:

Design database, UI screens, and system modules.

4. Coding & Unit Testing

Developers write code and test **individual modules**.

Example:

Coding login module and testing it separately.

5. Integration & System Testing

All modules are combined and tested as a **complete system**.

Example:

Testing login + payment + order tracking together.

6. Deployment

Software is **released to users**.

Example:

Launching the application.

7. Maintenance

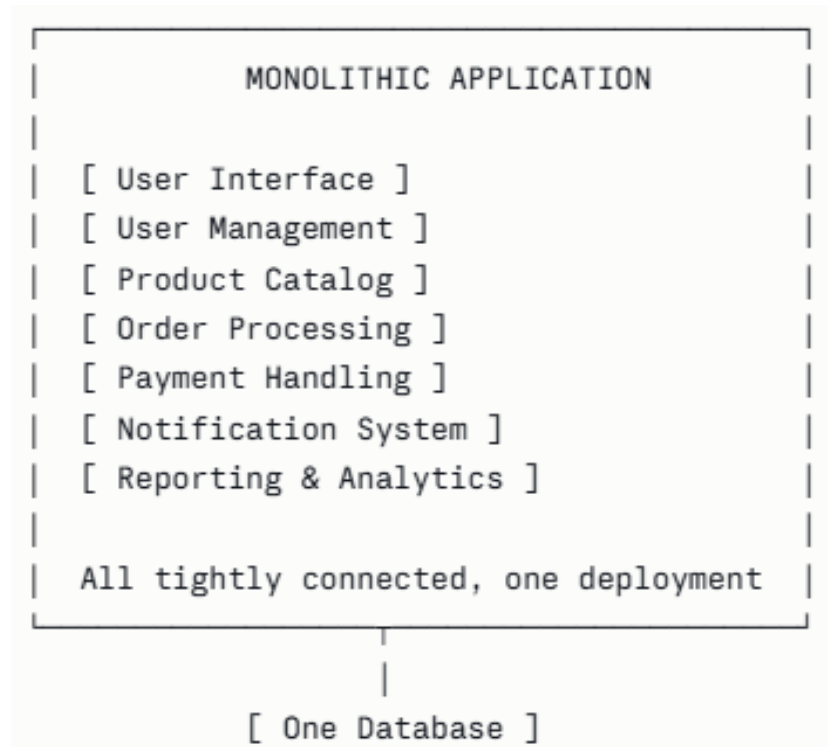
After release, bugs are fixed and improvements are made.

Example:

Fixing errors and adding new features over time.

7. Software Architectural styles (Any 1)

1. Monolithic Architecture



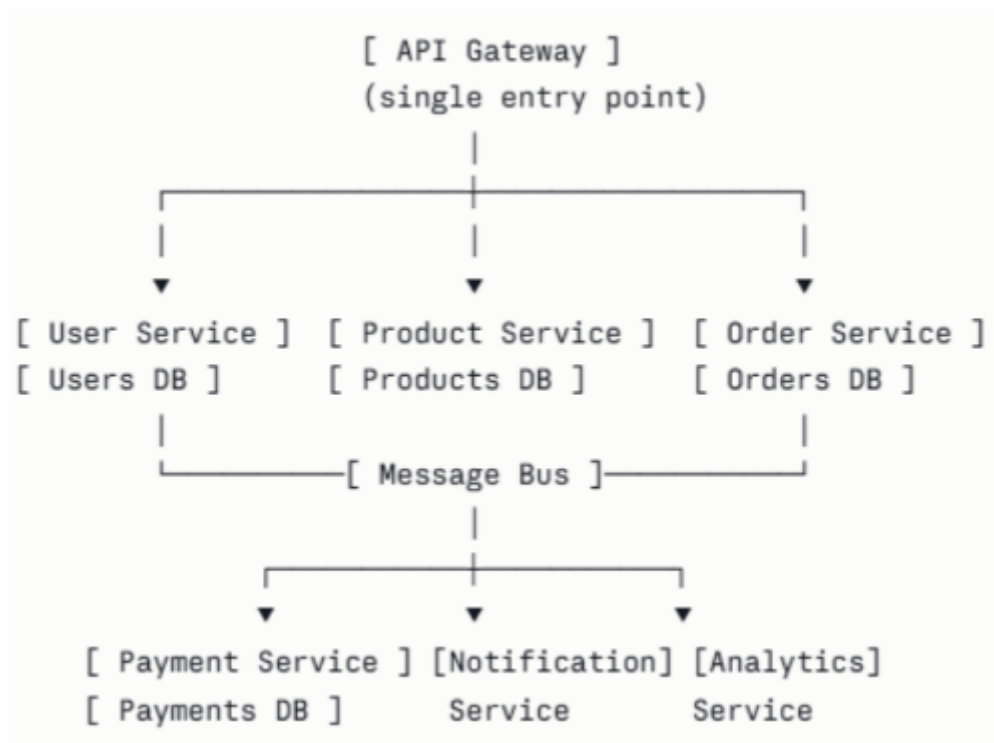
It is an architecture where the **entire application is developed as a single unified system** and all components are tightly connected.

- All modules (user, order, payment, etc.) are part of **one codebase and one deployment unit**.
- Easy to design and develop in the initial stages.
- Communication between modules is direct since everything is in one system.
- Difficult to scale because even a small change requires redeploying the whole application.
- Maintenance becomes complex as the system grows larger.

Example:

A food delivery app where all features (login, orders, payments) run as one application on a single server.

2. Microservices Architecture



It divides the application into **small, independent services**, each responsible for a specific function.

- Each service has its **own codebase, database, and deployment**.
- Services communicate through APIs (like REST or gRPC).
- Easy to scale specific parts of the system independently.
- Failure in one service does not affect the entire system.
- Increases complexity in managing multiple services.

Example:

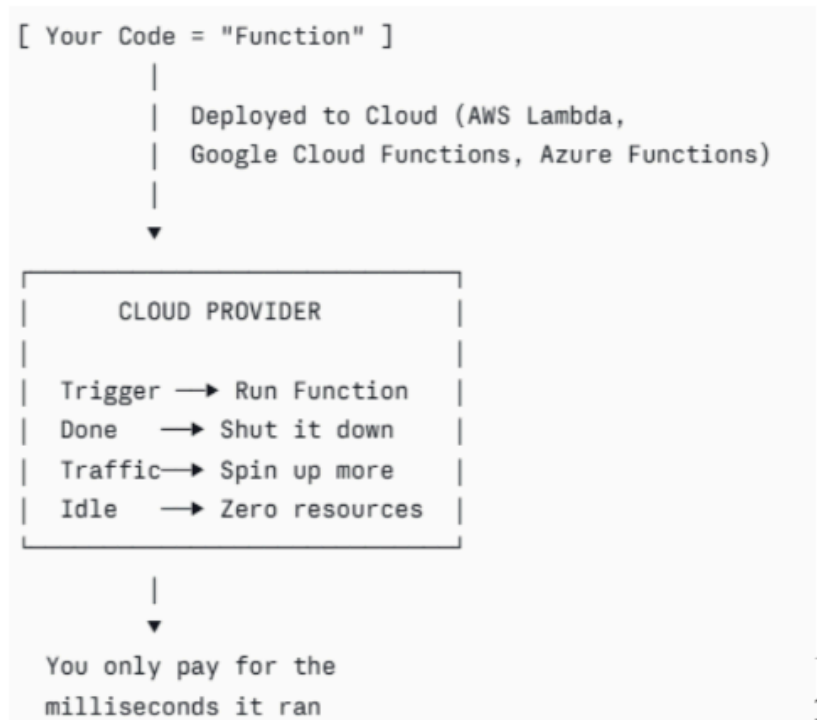
Food delivery app with separate services like User Service, Order Service, Payment Service, and Delivery Service.

3. Serverless Architecture

TRADITIONAL SERVER		SERVERLESS
You provision server	→	Cloud auto-provisions
You install OS	→	Cloud manages OS
You configure scaling	→	Cloud auto-scales
You pay 24/7	→	You pay per execution
You handle crashes	→	Cloud handles recovery
You manage security	→	Cloud manages patches

It is a cloud-based architecture where **developers do not manage servers**, and code runs on-demand.

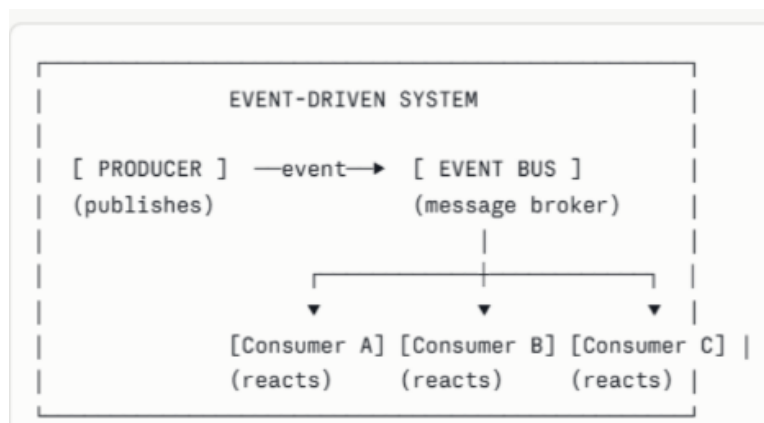
- Cloud provider handles infrastructure, scaling, and maintenance.
- Applications are executed as small functions triggered by events.
- Cost-effective because you pay only for actual usage.
- Automatically scales based on demand.
- Limited control over environment and may depend on cloud provider.



Example:

Using AWS Lambda to process user requests only when an event occurs (like uploading a file).

4. Event-Driven Architecture



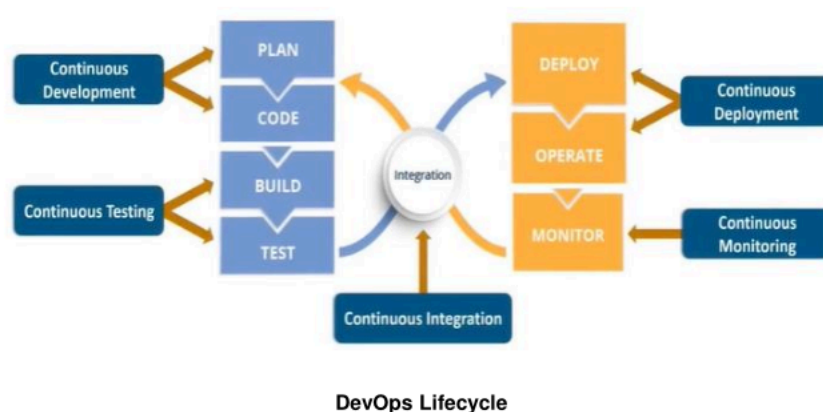
In this architecture, components communicate by **producing and consuming events**.

- Systems are loosely coupled, meaning components are independent.
- One component triggers an event, and others respond to it.
- Improves scalability and flexibility.
- Useful for real-time systems and asynchronous processing.
- Debugging can be complex due to indirect communication.

Example:

When an order is placed → event is generated → payment, notification, and delivery services respond automatically.

8.DevOps Life cycle (7 Stages)



1. Plan

Define requirements, features, and tasks for development.

Example:

Decide to add “live order tracking” in a food delivery app.

2. Develop

Write and manage code using version control systems.

Example:

Developers write code and store it in Git.

3. Build

Compile and convert code into executable form.

Example:

Generate application package (.jar, .exe).

4. Test

Automatically test the application to find bugs.

Example:

Run unit and integration tests

5. Deploy

Release application to production environment.

Example:

Upload app to server or cloud.

6. Operate

Maintain and manage the system in real-time.

Example:

Manage servers, handle user requests.

7. Monitor

Track performance and detect issues continuously.

Example:

Monitor app crashes, response time, user activity.